Experiment No. 4

Jenkins Installation and Basic Pipeline Creation

Aim : To install Jenkins, configure it for first-time use, and create a basic CI pipeline.

LO Mapped : LO3

Theory

Continuous Integration (CI)

Continuous Integration (CI) is a software development practice in which developers frequently merge their code into a shared repository. Every code change is automatically built and tested, allowing defects to be detected early. CI reduces integration issues, improves software quality, and speeds up software delivery.

Advantages of Continuous Integration

- Early detection of bugs
- Automated build and testing
- Faster software development
- Improved code quality
- Better collaboration among developers
- Reduced integration problems
- Faster release cycles

Jenkins

Jenkins is an open-source automation server written in Java that is widely used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates repetitive software development tasks such as compiling source code, running tests, generating reports, and deploying applications.

Jenkins supports thousands of plugins, allowing integration with Git, GitHub, Docker, Maven, Gradle, Kubernetes, cloud platforms, and many other development tools.

Features of Jenkins

- Open-source and free
- Cross-platform support
- Easy web-based interface

- Plugin-based architecture
- Supports CI/CD pipelines
- Distributed build system
- Integration with Git and other version control systems
- Automated testing and deployment

Requirements

- Java Development Kit (JDK)
- Jenkins WAR File
- Windows Operating System
- Web Browser
- Internet Connection (for downloading plugins)

Procedure

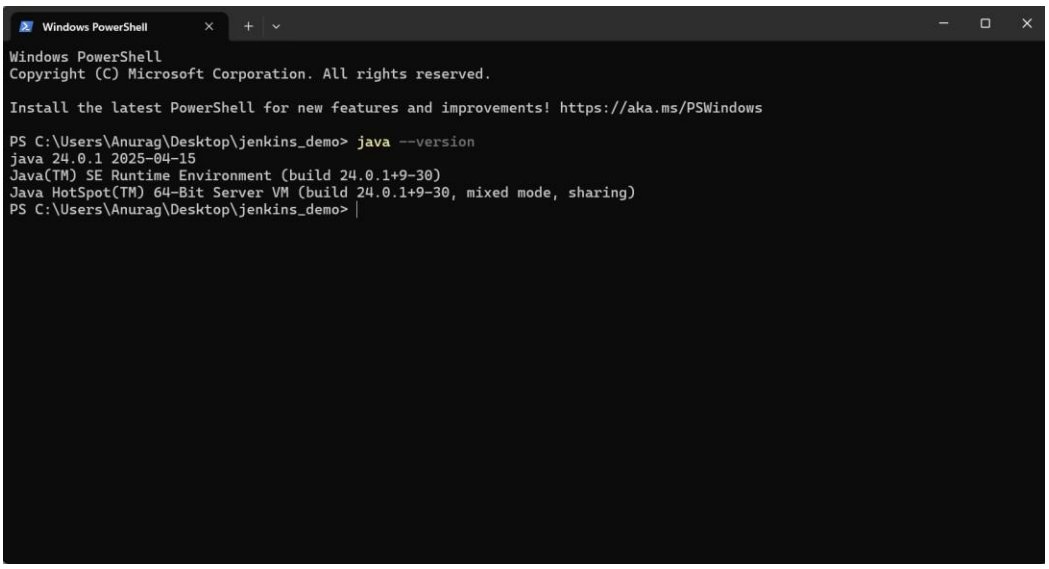
Step 1: Verify Java Installation

Before installing Jenkins, verify that Java is installed by opening Command Prompt or PowerShell and executing:

java -version

If Java is installed successfully, the installed JDK version will be displayed.

Figure 1: Java Installation Verification



Step 2: Launch Jenkins

Download the latest Jenkins WAR file from the official Jenkins website.

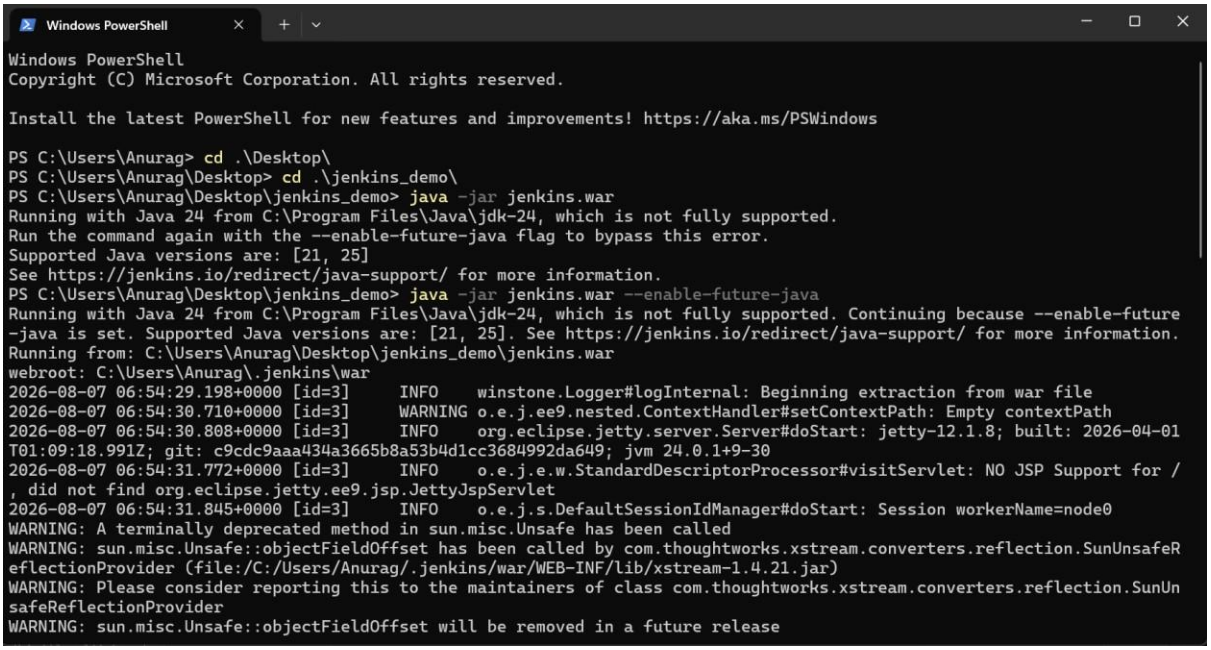
Navigate to the folder containing the WAR file and execute:

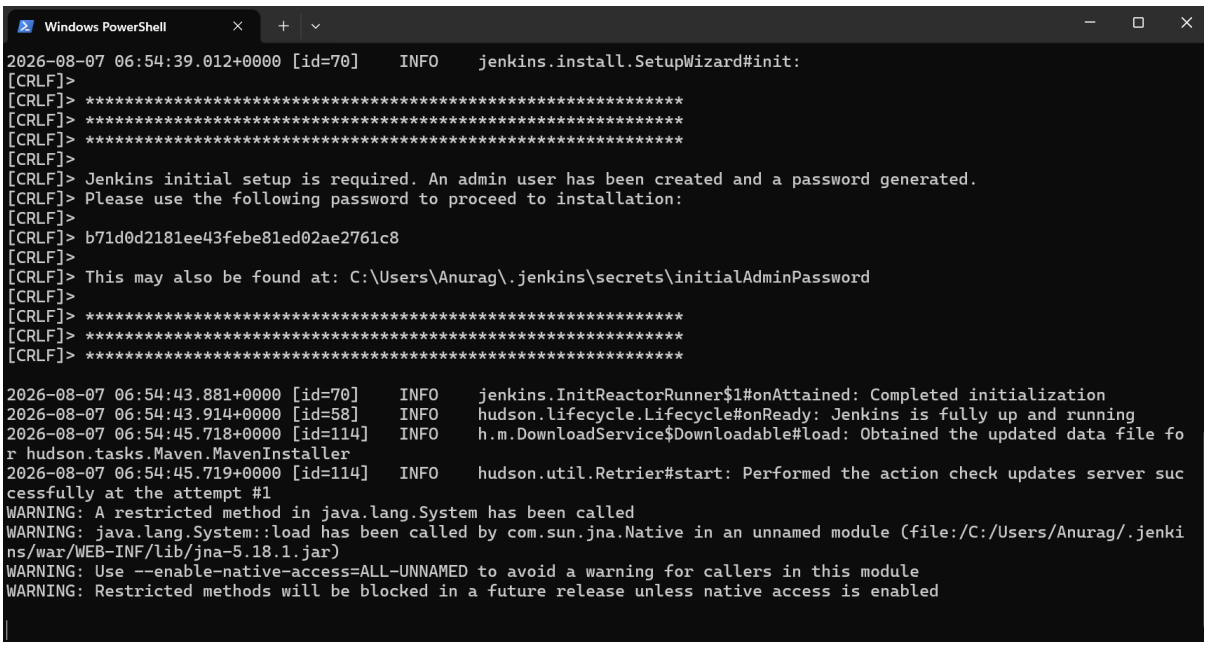
java -jar jenkins.war --enable-future-java

This command starts the embedded Jetty server and launches Jenkins on port **8080**.

The console displays logs indicating that Jenkins has started successfully.

Figure 2: Running Jenkins using WAR File





Step 3: Unlock Jenkins

When Jenkins starts for the first time, it generates an administrator password.

The password is available in the console or at:

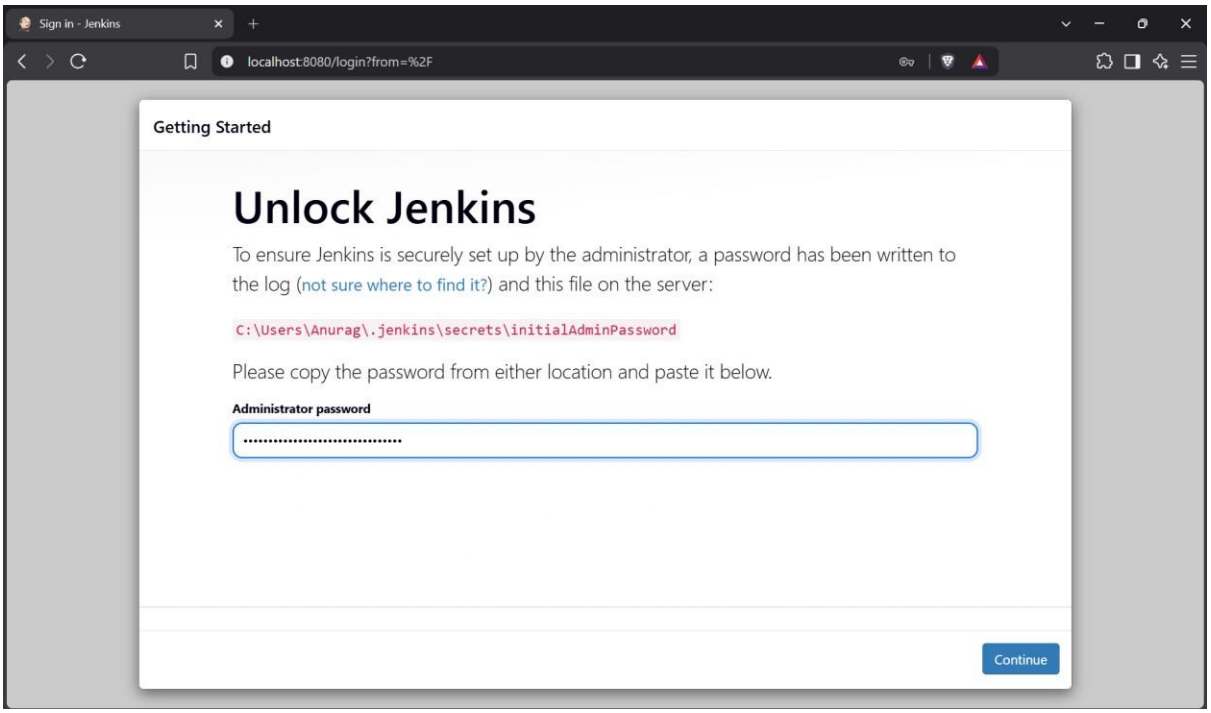
C:\Users\<username>\.jenkins\secrets\initialAdminPassword

Open the browser and visit:

http://localhost:8080

Enter the administrator password to unlock Jenkins.

Figure 3: Unlock Jenkins



Step 4: Install Suggested Plugins

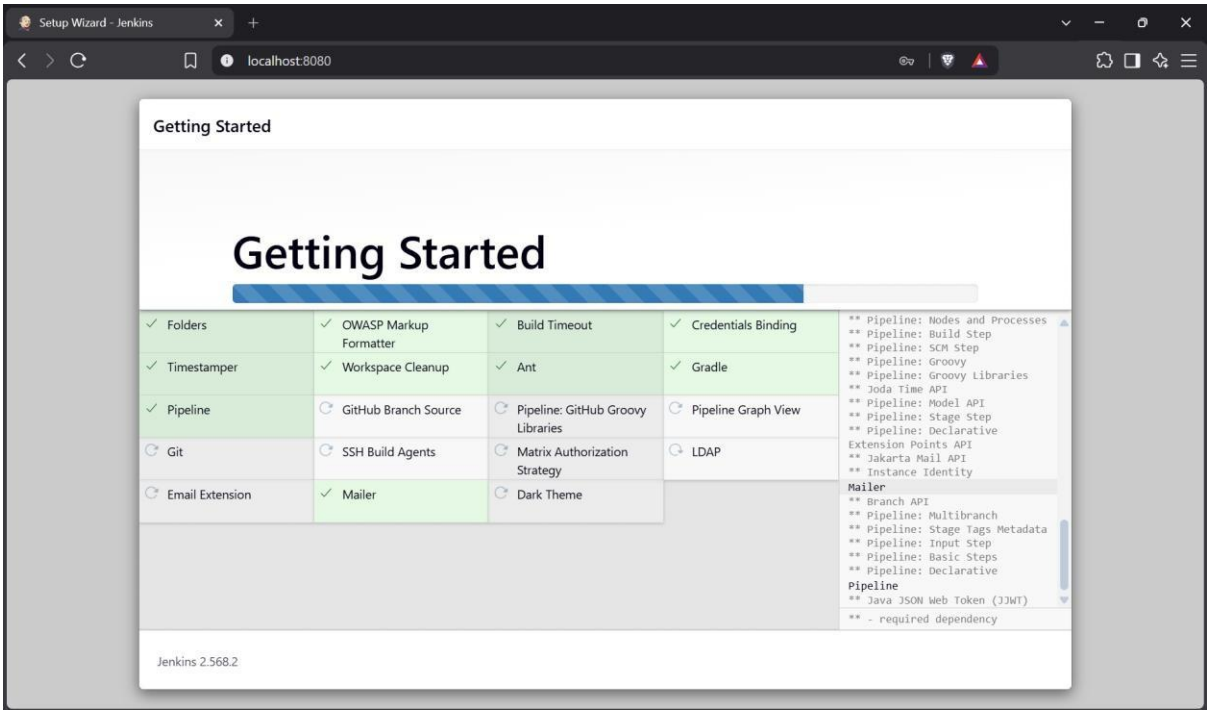
After unlocking Jenkins, select

Install Suggested Plugins

Jenkins automatically downloads and installs the most commonly used plugins required for building pipelines and integrating development tools.

Wait until installation is completed.

Figure 4: Plugin Installation



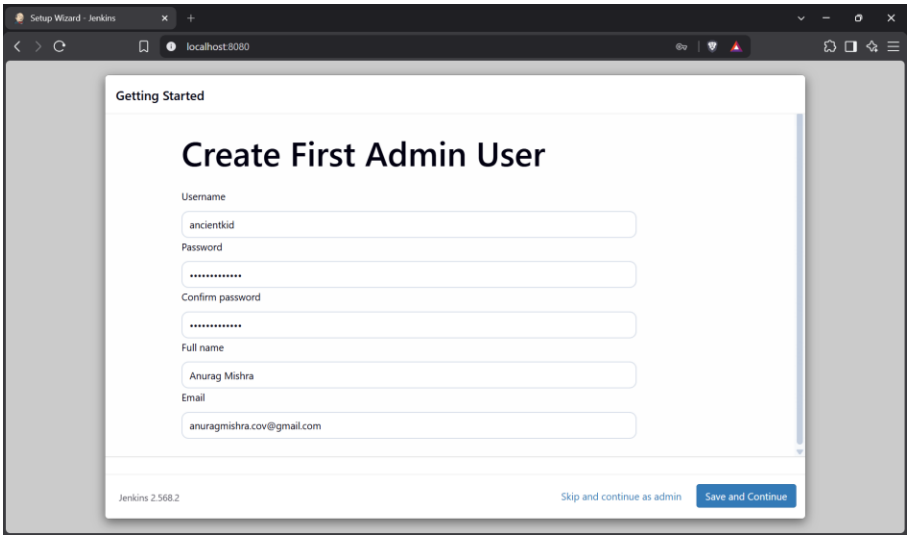
Step 5: Create First Administrator User

After installing plugins, create the administrator account by providing:

- Username
- Password
- Confirm Password
- Full Name
- Email Address

Click **Save and Continue**.

Figure 5: Create Administrator User



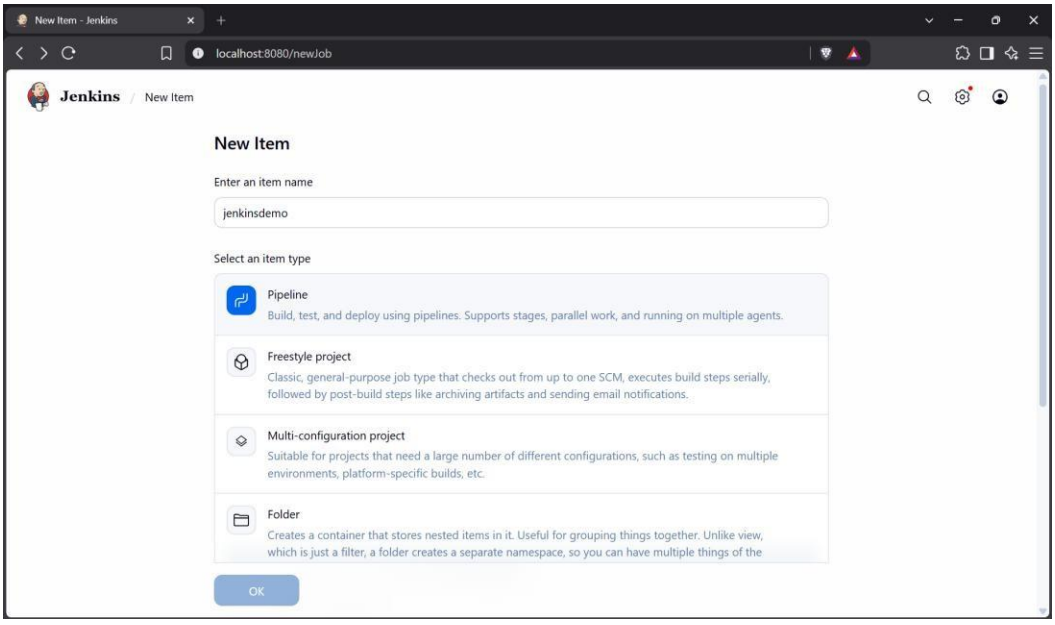
Step 6: Create a New Pipeline Job

After logging into Jenkins:

1. Click **New Item**
2. Enter the project name
3. Select **Pipeline**
4. Click **OK**

A new pipeline project is created.

Figure 6: Creating Pipeline Project



Step 7: Configure Pipeline Script

In the Pipeline section, choose

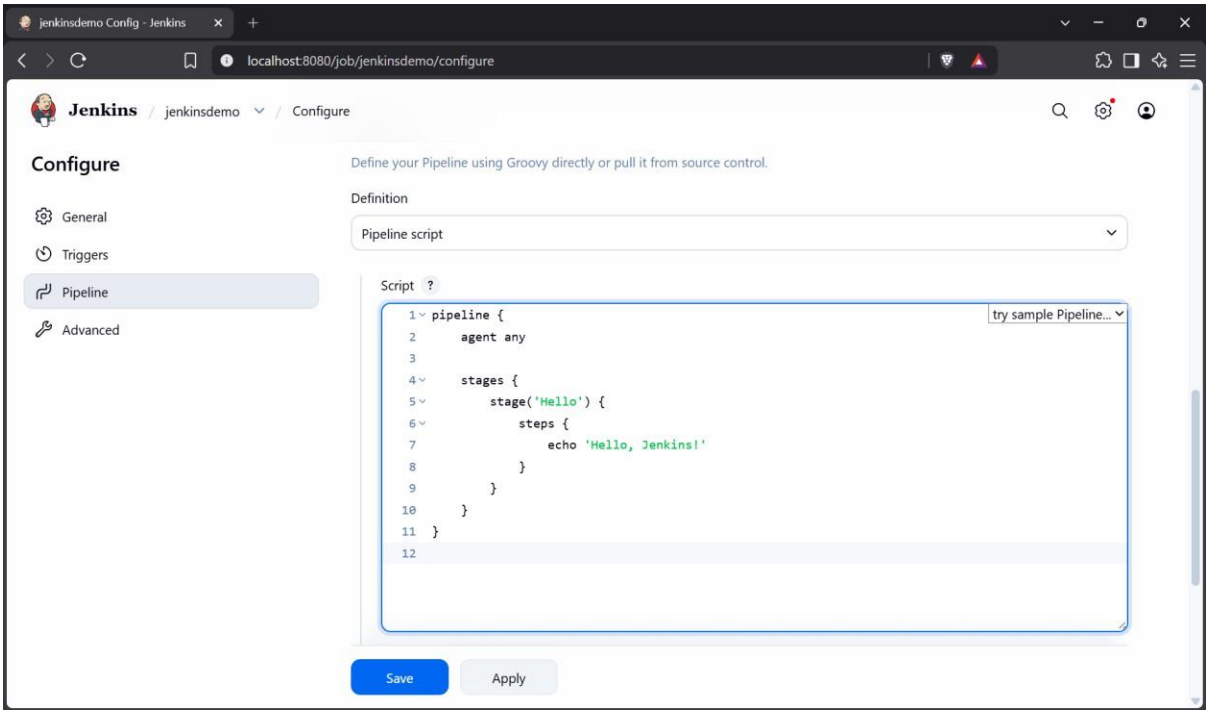
Pipeline Script

Enter the following script:

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins!'
            }
        }
    }
}
```

Click **Save**.

Figure 7: Pipeline Script Configuration



Step 8: Build the Pipeline

Click

Build Now

Jenkins executes the pipeline.

The console output displays:

Started by user

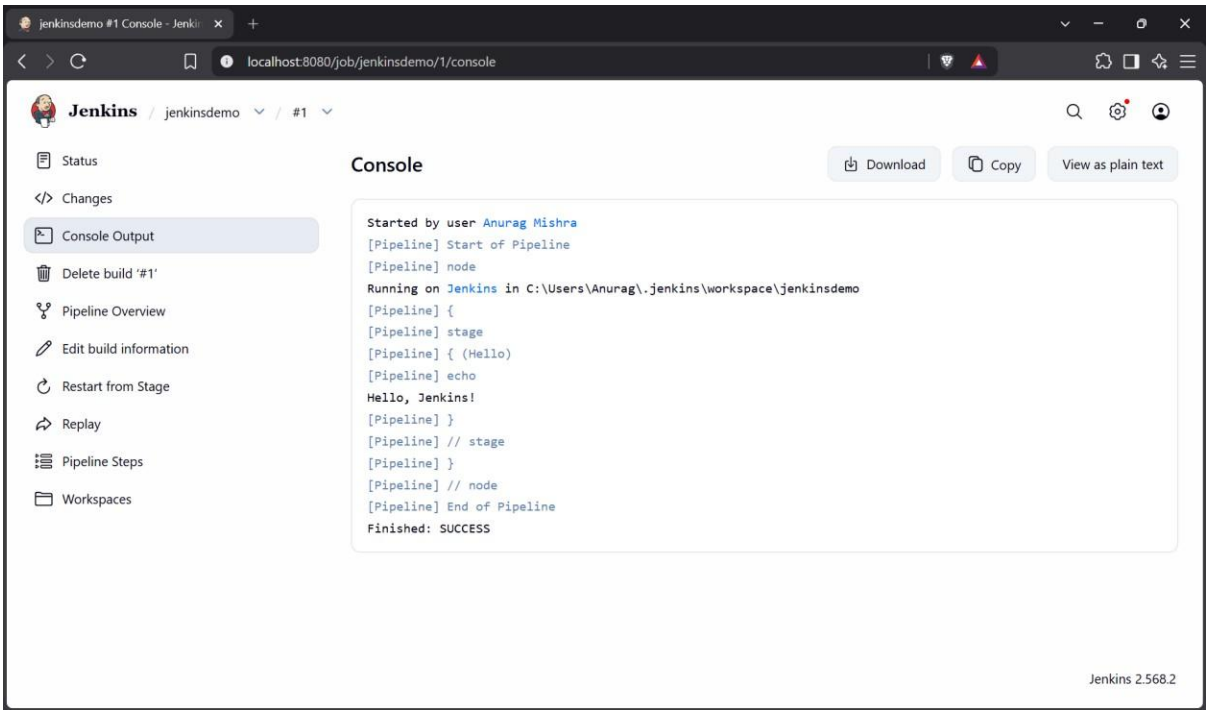
Running on Jenkins

Hello, Jenkins!

Finished: SUCCESS

indicating that the pipeline executed successfully.

Figure 8: Successful Pipeline Execution



Conclusion

The experiment successfully demonstrated the installation and configuration of Jenkins along with the creation of a basic pipeline. This experiment helped in understanding the fundamentals of Continuous Integration, Jenkins setup, pipeline creation, and automated execution of build steps.